App States and Considerations for HW3 - State Management
This document outlines the key app states our single-page text entry app can enter on the Android platform, along with considerations and actions taken in each state to preserve the text entered by the user. The purpose is to ensure a reliable, user-friendly experience where users can confidently interact with the app without losing any input data.

Tombstone (Suspend/Resume) Management: The use of SavedStateHandle in TextBoxViewModel effectively handles the "tombstone" state by preserving the contents of the TextField even if the app is destroyed and restarted by the system. The app will demonstrate that the user can leave, return, and see the same text they entered in the TextField.

1. Active State
- **Description**: The app is in the foreground and receiving user input.
- **Considerations**:
  - Real-time updates to the text field as the user types to enhance the interactive experience.
  - Responsive validation for text input to maintain the app's usability and integrity.
- **Actions Taken**:
  - Continuously updates the text field with user input.
  - The text is managed with SavedStateHandle to allow seamless recovery if the app moves to other states.

2. Inactive State
- **Description**: The app remains in the foreground but is not actively receiving user input, such as when a dialog appears or the user switches to another app briefly.
- **Considerations**:
  - The text should be preserved to provide a seamless return experience for the user.
- **Actions Taken**:
  - Text entered is saved with SavedStateHandle, so if the app remains inactive for a brief period, user input is maintained and visible upon returning.

3. Background State
- **Description**: The app is fully in the background and not visible to the user but is still running.
- **Considerations**:
  - Efficient use of memory and processing power to avoid battery drain or unexpected app termination.
- **Actions Taken**:
  - Data is preserved in SavedStateHandle as the app moves to the background. This approach minimizes resource use while retaining the text entry content. Additional optimizations such as limiting network usage and intensive processing could be considered in future expansions of the app.

4. Suspended State (Tombstoned State)
- **Description**: The app may be temporarily moved out of memory but is still managed in the device's background memory, ready to resume when the user returns.
- **Considerations**:
  - Ensure that critical app state is preserved to support a seamless user return.
- **Actions Taken**:
  - The text entered is saved with SavedStateHandle, allowing the app to restore the exact state of the text field when reactivated, enabling the user to continue without disruption.

5. Terminated State
- **Description**: The app is closed and not running in memory, either due to user action or system management under memory pressure.
- **Considerations**:
  - Prevent data loss, as the app may be reopened by the user at any point, expecting their work to preserved.
- **Actions Taken**:
  - Data is stored using SavedStateHandle or SharedPreferences as a backup. This ensures that even be if the app is terminated, the user's text input can be restored upon reopening, offering a consistent experience.

6. Restart State (Cold Start)

- **Description**: The app is launched from scratch, either after being terminated or after a device restart.
- **Considerations**:
  - Minimize startup time and initialize the app with user input preserved for continuity.
- **Actions Taken**:
  - Upon launch, the text input is restored from SavedStateHandle (or SharedPreferences if necessary) so that users can pick up where they left off. The app is designed to initialize quickly, making user interaction feel immediate.

Specific States Considered for My App:

- **Paused**: When the user switches to another app, onPause() is called. The app should retain any unsaved changes in the text box, so it doesn't need to reset if the user navigates back to it.
- **Stopped**: When the app is backgrounded for a longer time, onStop() may be called. The SavedStateHandle retains data across stops to ensure the app retains text when resumed.
- **Destroyed**: If Android needs to reclaim resources, it may destroy the app. SavedStateHandle is used here to save the state across such instances.

Why Preserving Text is Necessary for a Good User Experience

Preserving the text entered by the user across different states enhances the user experience in the following ways:

- **Continuity**: Users often switch between apps. The ability to resume typing in the text field without losing data allows for a seamless workflow and reduces frustration.
- **Reliability**: By preserving the input in critical states like Paused, Stopped, and Destroyed, users can trust the app to save their progress and minimize the risk of data loss.
- **Usability**: An app that retains user data across various states and interactions provides a more robust, user-friendly experience, especially in cases where data entry is interrupted.

This approach ensures the app delivers a stable experience across the different stages of the Android Activity Lifecycle while prioritizing resource efficiency and data integrity.